Embedded Systems Assessment 3 Report FreeRTOS Implementation

1. Use of the FreeRTOS API

1.1 Tasks

Tasks are spawned in the setup() using the function:

```
xTaskCreatePinnedToCore(TaskA,"TaskA",4096,NULL,6,NULL,app_cpu);
```

This instruction creates the task and attributes to it the function the task must execute, the name of the task, a specified amount of RAM, a pointer to data to be passed to the task , a priority level, the handle of the task, and a CPU core to execute on. Each task is given its own function. The lowest priority task is given a priority of 1 and the highest has a priority of 6. The tasks and the program will execute on app_cpu which is the second core.

1.2 Synchronization

Interrupts, semaphores, and mutexes are used to control when task execution begins. All interrupts execute when brought from LOW to HIGH. Binary semaphores are created in the setup(). At the beginning of the program no tasks execute until SYNC is pressed. All the tasks wait for a SYNC semaphore before starting. When a SYNC interrupt occurs, a binary semaphore is given and the task execution begins. portMAX_DELAY) forces the tasks to wait until the sync Semaphore is given.

```
xSemaphoreTake(syncSemaphore, portMAX_DELAY);
```

Task S also waits for an sSemaphore before it can execute, which is given by the S input pulse ISR.

A mutex is used in tasks A, B, and AGG, to prevent the three tasks from reading or writing to the countA or countB values while one of the others is doing so.

```
void TaskA(void *pvParameters)
{
  xSemaphoreTake(syncSemaphore, portMAX_DELAY); //waits for SYNC pulse
  xSemaphoreGive(syncSemaphore);   //returns semaphore for other tasks to use
  TickType_t xLastWakeTime = xTaskGetTickCount();

  while(1)
  {
    if (xSemaphoreTake(xTokenMutex, portMAX_DELAY) == pdTRUE) //check mutex if
available
    {
      g_monitor.beginTaskA(IDA);
```

```
      digitalWrite(ACK_A, HIGH);
      uint16_t countA_total = countA;
      countA = 0;
      uint32_t seed = (IDA << 16) ^ countA_total ^ 0xA1;
      tokenA = WorkKernel(672000, seed);
      xSemaphoreGive(xTokenMutex);    //give mutex back
      Serial.printf("A,%d,%d,%d\n",IDA,countA_total,tokenA);
      ++IDA;
      digitalWrite(ACK_A, LOW);
      g_monitor.endTaskA();
    }
    vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(10)); // Exact 10ms period
  }
}
```

As can be seen in the above example code, the mutex is checked and if it is available it is taken, preventing the other tasks from reading or writing to the count A and B variables. The task does the math that it needs to do, then gives it back.

1.3 Timing

```
vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(10));
```

This function pauses only the specific task it is used in, putting the task on hold during the duration. Its used in this code to create precise deadline timing periods for tasks.

1.4 ISRs

```
xSemaphoreGiveFromISR(sSemaphore, &xHigherPriorityTaskWoken);
if (xHigherPriorityTaskWoken) portYIELD_FROM_ISR();
```

This is instruction was used in both ISRs to give semaphores to sSemaphore and syncSemaphore. xHigherPriorityTaskWoken and portYIELD_FROM_ISR(); are use to start task S immediately after the ISR is called.

2. Task Priorities

The execution time of each task was calculated using the period of the CPU clock and the budget cycles handed to the work kernel for each individual task. Calculation results were placed into a table with the period of each task's deadline.

$$Formulas:$$

$$T_{CPU} = \frac{1}{f_{CPU}}$$

$$t_{task\ duration} = T_{CPU} * B_{task\ budget}$$

$$Example\ calulation, where: f_{CPU} = 240MHz, B_{task\ budget\ A} = 672000\ cycles$$

$$T_{CPU} = \frac{1}{240000000} = 4.167ns$$

$$t_{task\ duration\ A} = 4.167ns * 672000 = 2.8ms$$

Table 1: Task deadline periods, budgets, and durations.

Task	Dealine Period (ms)	Budget (cycles)	Total Compute time (ms)
A	10	672000	2.8
B	20	960000	4
AGG	20	480000	2
C	50	1680000	7
D	50	960000	4
S	30	600000	2.5

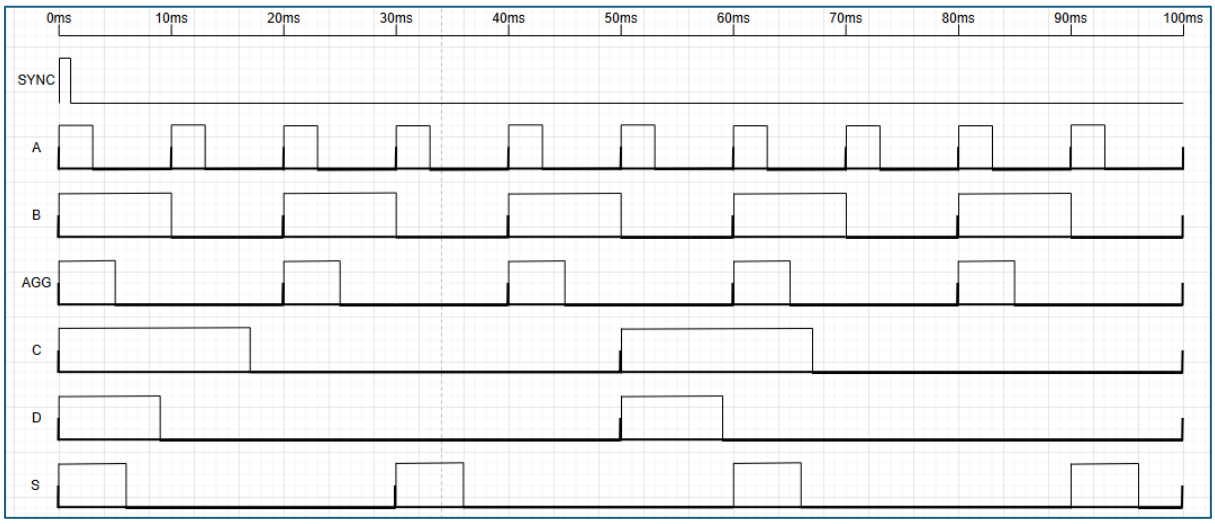

Figure 1: SYNC and task releases, deadlines, and durations visualised using Draw.io.

It should be noted that compute time is simply the approximate duration of the work kernel function execution for each task. Compute time as calculated in Table 1 doesn't account for the time taken to execute each task's other instructions. Most of the Arduino IDE instructions ESP32

execute in a matter of several nano seconds. Compute time estimates are therefore likely too short by 1% or so. In the cyclic executive version of this program, this was accounted for by giving each task some “slack”, by increasing the hypothetical length of each task in the scheduler by 0.1ms, providing enough time for each task’s execution.

Although S isn’t cyclical, it was included above, appearing cyclically to illustrate a potential execution of S, given it can only occur once every 30ms.

A graphic design software called Darw.io was used to visualize the task releases, deadlines, and durations. This diagram shows Task A is by far the most likely to miss its deadline as its released most frequently, and possesses the highest duration time to deadline period ratio of any of the tasks. Following this logic implies that tasks with the shorter deadline periods and low duration to deadline period ratios should take higher priorities. However, if a task has a dependency on the output of another task, the task that outputs the dependency should be prioritised over the task that requires the dependency. Given this logic, task were prioritised as so (1 being lowest and 6 being highest):

1. Task S – This is being assessed on its ability to complete within soft time rather than hard time, giving it more flexibility to be executed when no other tasks are busy.
2. Task D – Tasks D and C share the lowest deadline frequency of all the tasks.
3. Task C – This task has a longer duration than D, giving it a higher duration to deadline period ratio. This means that although C and D have the same deadline period, C has to do more to complete itself in that time, making its execution more urgent.
4. Task AGG –Tasks B and AGG have the same deadline period, but AGG’s output depends on the output from both A and B, meaning that both A and B must have executed before this task.
5. Task B – B must execute before AGG and has a longer deadline period than A.
6. Task A – This task has the highest release and delaine frequency and is therefore given the highest priority.

3. Stack Sizes

The final version of the code uses the task create functions:

```
xTaskCreatePinnedToCore(TaskA,"TaskA",4096,NULL,6,NULL,app_cpu);
xTaskCreatePinnedToCore(TaskB,"TaskB",4096,NULL,5,NULL,app_cpu);
xTaskCreatePinnedToCore(TaskAGG,"TaskAGG",4096,NULL,4,NULL,app_cpu);
xTaskCreatePinnedToCore(TaskC,"TaskC",4096,NULL,3,NULL,app_cpu);
xTaskCreatePinnedToCore(TaskD,"TaskD",4096,NULL,2,NULL,app_cpu);
xTaskCreatePinnedToCore(TaskS,"TaskS",4096,NULL,1,NULL,app_cpu);
```

These include 4096 bytes (approx. 4kB) of RAM for each task. In earlier versions of this code, only 1024 bytes (approx. 1kB) was allocated to each task. When this was uploaded to the ESP with the SYNC pin connected to ground, the following report was produced every second:

```
----REPORT----
A jobs=0 misses=0 max=0 late=0
B jobs=0 misses=0 max=0 late=0
AGG jobs=0 misses=0 max=0 late=0
C jobs=0 misses=0 max=0 late=0
D jobs=0 misses=0 max=0 late=0
S jobs=0 misses=0 max=0 late=0
```

Which indicates that no tasks are executing, which is what is expected as SYNC hasn't been brought HIGH yet. Once SYNC is brought HIGH, the following error messages repeating over the serial monitor:

```
Rebooting...
ets Jul 29 2019 12:21:46

rst:0xc (SW_CPU_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4640
load:0x40078000,len:15620
ho 0 tail 12 room 4
load:0x40080400,len:3164
entry 0x4008059c
```

This indicates that the ESP is repeatedly rebooting once tasks are expected to execute.

To troubleshoot the issue, the stack size available for each task was increased from 1kB to 4kB, as it was believed that the crash might be due to memory heavy functions like serial communication operating in tasks with too little stack space, causing stack overflow and continues crashes.

When the stack size for each task was increased the repeating crashes ceased, and the code operated as expected, indicating that sufficient stack space had been allocated.

4. Comparison Between FreeRTOS and Cyclic Executive Implementations.

4.1 Comparison

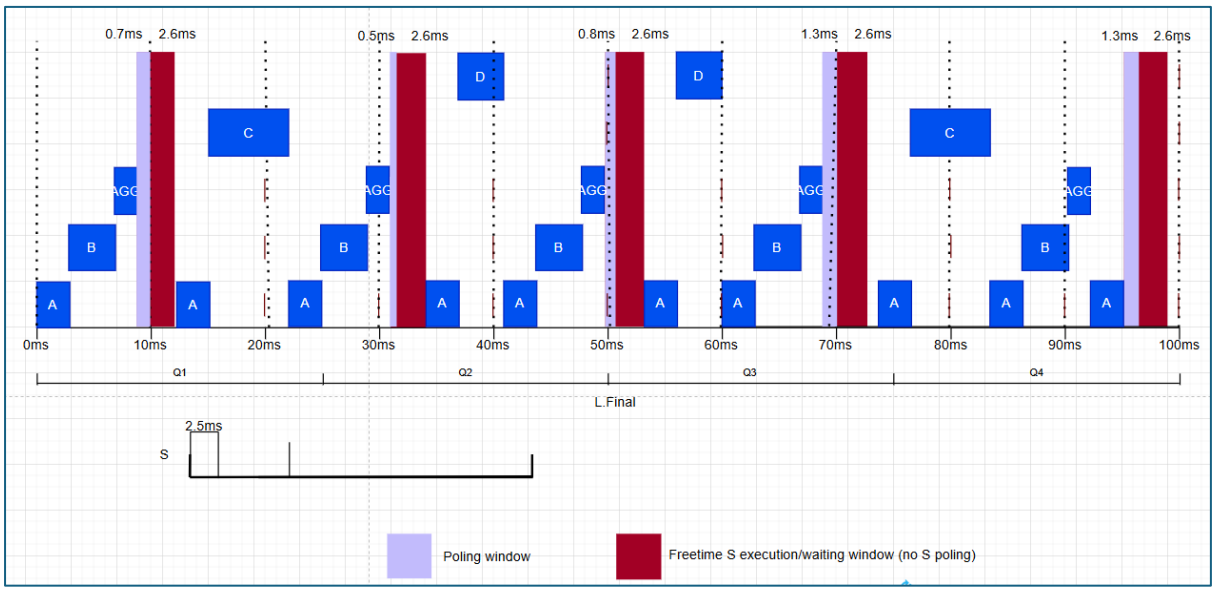

Figure 2: Visualisation of the cyclic executive version schedule.

The cyclic executive program was scheduled by creating boxes with the length of each task (actually each task +0.1ms slack) in Draw.io, then arranging them end to end in a 100ms hyperloop, as it was noticed that all the task deadlines aliased at 100ms. It was ensured that any 30ms period within the hyperloop contained time to poll an SCalled flag (the polling window) and time to execute task S (Freetime). Excel was then used to calculate the precise execution time of each task. Functions to execute each task were written and a struct array was used to create a timing table. The main loop would check the current microsecond compared to T0, against the time member of the current task in the struct array. Once the two match, the task current would execute, and the main loop would check the next task.

During testing, a signal generator was used to produce countA and countB signals, buttons were used to send SYNC signals, and to toggle mode. The monitor results, show that the cyclic executive version of the program was much more reliable at meeting deadlines than the FreeRTOS program. This is likely for a few reasons:

- In the FreeRTOS version, for example, when A task needs to execute, all other tasks make way for it by stopping and allowing A to execute, as A has the highest priority. Tasks getting done only in pieces as they are constantly paused for higher priority task might compound into tasks not being done on time, especially when multiple interrupts are used.
- Time between finishing or pausing a task and starting the next one isn't instant. Small amount of time is taken to do this, and constant context switching can compound into significant total time when the OS isn't actually executing a task.

The FreeRTOS program's reliability could be improved by using more FreeRTOS instructions.

Table 2: Cyclic executive version schedule in Excel.

Order	Task	time
0	A	0
1	B	2900
2	AGG	7000
3	POL	9100
4	FREE	9800
5	A	12400
6	C	15300
7	A	22400
8	B	25300
9	AGG	29400
10	POL	31500
11	FREE	32000
12	A	34600
13	D	37500
14	A	41600
15	B	44500
16	AGG	48600
17	POL	50700
18	FREE	51500
19	A	54100
20	D	57000
21	A	61100
22	B	64000
23	AGG	68100
24	POL	70200
25	FREE	71500
26	A	74100
27	C	77000
28	A	84100
29	B	87000
30	AGG	91100
31	A	93200
32	POL	96100
33	FREE	97400

4.2 Advantages vs disadvantages

Table 3: Advantage/disadvantage table.

	Cyclic executive	**FreeRTOS**
Advantages	• Deterministic execution produces predicable behavior. • Only makes use of standard C programming libraries like stdbool.h. • Program flow is more apparent improving readability.	• Potentially shorter development time as the RTOS manages scheduling. • Allows single core "multitasking" that appears simultaneous to humans. • Blocking inactive tasks rather that polling variables to know when to execute tasks is more efficient CPU usage.
Disadvantages	• Longer development time for scheduling. • No multitasking means program counter may be stuck in a loop polling, wasting cycles.	• Can be harder to debug due to its non-deterministic behavior. • Relies on external non-standard libraries. • Potentially slower input response times due to task having to be "put down" before another can be "picked up".